Model Serving: REST, gRPC, and Streaming

Key Takeaways

Production is many models, not one

Netflix uses ML for 20+ tasks. A ride-sharing app alone may run 200 models across markets. Your infrastructure decisions must assume many models, independent update schedules, and concurrent traffic — not a single notebook inference call.

Online and batch are not the same as synchronous and asynchronous

Online prediction happens in real time (user is waiting). Batch prediction is pre-computed (user fetches a stored result). A model can serve online predictions using batch-computed features. Do not conflate the serving mode with the feature freshness.

Measure latency in percentiles, not means

Mean latency hides tail behaviour. At 1M QPS, a p99 of 4,000ms means 10,000 users per second experiencing a broken product. Always instrument p50, p95, and p99. Set your SLA on p99, not on the average. The Akamai and Booking.com studies prove this directly costs revenue.

Deployment strategy is risk management

Shadow deployment proves correctness before any user is affected. Canary deployment limits blast radius during rollout. Blue-green enables instant rollback. A/B testing provides causal evidence of business impact. Choose your strategy based on risk tolerance and what you need to prove.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

Q1. A model was deployed six months ago with no monitoring and no retraining. Business metrics have been quietly declining. What Chip Huyen myth does this scenario directly contradict, and what two things should have been in place from the start?

Q2. Your model currently serves 100 QPS. A new dynamic batching configuration groups 20 requests and processes them in 18ms. The old approach processed 1 request in 2ms. (a) What is the new throughput in QPS? (b) What is the new latency per individual request? (c) Is this a good trade-off for a user-facing product? Why or why not?

Q3. You are deploying a new fraud detection model. Explain in one sentence each what the following strategies would look like in this specific context: (a) shadow deployment, (b) canary deployment, (c) blue-green deployment.

Q4. What is the difference between a streaming feature and an online feature? Give one concrete example of each for a ride-sharing surge pricing model.

Coding / Design Exercises

Work through these independently or in pairs. These are open-ended — focus on reasoning, not a single right answer.

Exercise 1: Wrap a Model with BentoML

Take any sklearn or PyTorch model you trained in a previous course. Using the BentoML documentation (`docs.bentoml.com`), wrap it in a simple REST service that accepts JSON input and returns a prediction. Run it locally. Measure the latency of 100 sequential requests using Python's `time` module and compute the p50 and p99. Write one paragraph describing what you observed and what you would change to improve the p99.

Exercise 2: Latency Budget Analysis

Pick a user-facing ML application you use regularly (e.g. Google Search, Spotify recommendations, Instagram feed). Estimate the *total latency budget* available for ML inference: (a) What is the acceptable end-to-end response time from a user perspective? (b) How much of that budget is consumed by network, preprocessing, and postprocessing? (c) What is left for the model itself? Sketch a diagram showing the latency breakdown and argue for your estimates.

Exercise 3: Prediction Mode Decision

For each of the following use cases, decide: (1) batch prediction, (2) online prediction with batch features, or (3) online prediction with streaming features. Justify each answer in one sentence.
(a) Monthly customer churn scores for a SaaS product.
(b) Real-time spam filter for an email server.
(c) Dynamic ad bidding that uses a user's last 3 clicks.
(d) Personalised homepage recommendations shown at login.

Challenge Problem

Design a Serving Architecture for a Recommendation System

You are the ML engineer at a streaming platform with 5 million daily active users. The product team has defined a strict SLA: recommendations must load within 100ms of page open. Design the full serving architecture. Your design must address: (1) Which prediction mode will you use and why? Justify with the 100ms SLA. (2) Which features are batch features and which are streaming features? Give at least two examples of each. (3) How will you deploy a new recommendation model next quarter — which deployment strategy and why? (4) What three metrics will you include in your SLO, and at which percentile? (5) What happens to recommendations when your serving infrastructure goes down — what is your fallback? One paragraph per question, or a structured design doc format.

Further Reading

- **Chip Huyen — Designing Machine Learning Systems, Ch. 7** — O'Reilly, 2022 (course primary text)
  "Model Deployment and Prediction Service." The four deployment myths, batch vs. online prediction, and the three prediction modes we covered today all come directly from this chapter. Essential reading.
- **Chip Huyen — Designing Machine Learning Systems, Ch. 1** — O'Reilly, 2022
  The latency vs. throughput discussion and the percentile argument are covered here. Read alongside Ch. 7 for the full picture.
- **BentoML Documentation** — docs.bentoml.com
  Start with the "Getting Started" guide. The serving and batching sections are directly relevant to Exercise 1. The deployment guide covers Docker and Kubernetes packaging.
- **NVIDIA Triton Inference Server Docs** — docs.nvidia.com/deeplearning/triton-inference-server
  Focus on the dynamic batching and model ensemble sections. The performance analyser tool is worth knowing for production tuning.

Online Resources

- **Chip Huyen's Blog** — huyenchip.com — real-world ML serving war stories
- **Akamai 2017 Study** — search "Akamai 100ms conversion rate" — the 7% figure we cited
- **Booking.com Latency Study** — "Booking.com 2019 latency experiment" — 0.5% latency cost
- **Martin Fowler — Blue-Green Deployment** — martinfowler.com — the canonical reference
- **Google SRE Book, Ch. 4** — sre.google/sre-book — SLOs and SLAs explained with production examples
- **The Morning Paper** — blog.acolyer.org — academic ML systems papers in plain English

Answers

A1. This contradicts **Myth 2** ("if we do nothing, performance stays the same") and **Myth 3** ("we won't need to update much"). From the start: (1) a monitoring pipeline tracking input feature distributions and business metrics at serving time, with automated drift alerts; (2) a scheduled retraining pipeline (or drift-triggered retraining) to keep the model up to date with the current data distribution.

A2. (a) New throughput: $20/0.018 \approx \mathbf{1{,}111}$ QPS. (b) New latency per request: 18ms (all requests in the batch wait for the batch to complete). (c) For a user-facing product, this depends on the SLA. If users can tolerate 18ms (e.g. a background API call), it is a good trade-off — throughput improved 11× vs. sequential single-request processing. If users expect <5ms response time (e.g. real-time UI), the 18ms batch latency may violate the SLA. The batching paradox only applies when the latency increase is within your budget.

A3. (a) **Shadow**: run the new fraud model in parallel with the current model, log both predictions for every transaction, compare offline — no user ever receives the new model's decision. (b) **Canary**: route 5% of live transactions to the new fraud model, monitor false positive and false negative rates on real traffic, increase percentage if stable. (c) **Blue-green**: stand up a complete new serving environment running the new fraud model, run both environments simultaneously, switch 100% of transaction traffic to the new environment at once, keep the old environment warm for instant rollback.

A4. An **online feature** is any feature used at online prediction time — it can be a pre-computed batch value fetched from a data warehouse. A **streaming feature** is specifically computed from a live event stream in real time. Example for surge pricing: "average ride demand in this area over the past 30 days" (updated nightly from historical data) is an online feature that is also a batch feature. "Number of ride requests in this area in the last 5 minutes" (computed from a live Kafka event stream) is a streaming feature.